

Queues

DSA27 — Lecture 07 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Faculty of Computers and Information Sciences, Mansoura University

Week 7 · Fall 2026

Where We Are

One end was easy. Now we need both.

Week 6: the **stack** touches **one** end. Put the top at the cheap end, done.

Today: the **queue** adds at one end and removes at the **other**.

- A dynamic array is cheap at the **end** only.
- A singly linked list is cheap at the **head** (and, with a tail, for adding at the end).

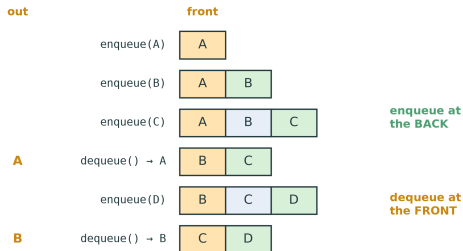
So which end goes where, and what happens to the expensive end?

1. The Queue ADT: enqueue, dequeue — FIFO
2. The obvious design, `SlowQueue`, and its $O(n)$ dequeue
3. The ring buffer: `CircularQueue`, every operation $O(1)$
4. Full or empty? — and growing a ring without scrambling it
5. The linked queue, and a queue made of two stacks
6. Measured: the gap is the lesson
7. Where queues are used — and the deque

The Queue ADT

First in, first out

A queue: in at the back, out at the front — first in, first out (FIFO)

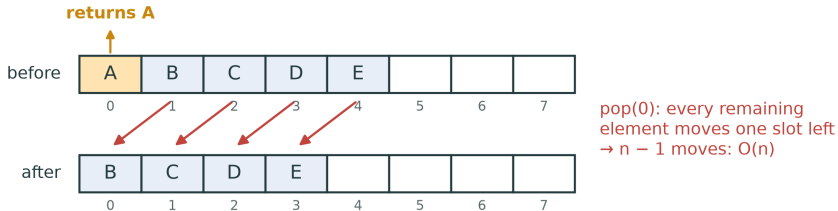


Operation	Meaning	Cost
enqueue(x)	add x at the back	$O(1)$
dequeue()	remove and return the front (IndexError if empty)	$O(1)$
is_empty(), len(q)		$O(1)$

The Obvious Design

SlowQueue: a DynamicArray, as it is

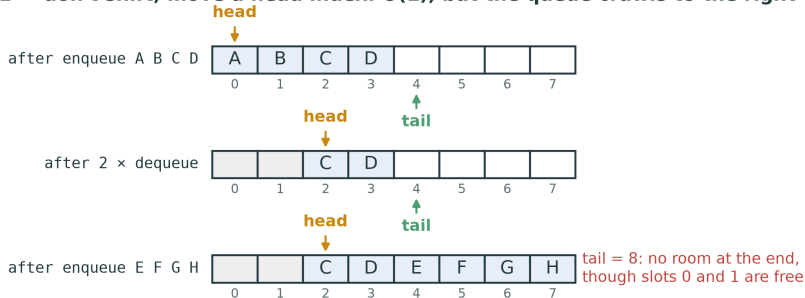
SlowQueue: enqueue is append ($O(1)$); dequeue is $\text{pop}(0)$ — $O(n)$



- enqueue = append at the end: amortised $O(1)$.
- dequeue = $\text{pop}(0)$: every remaining element shifts left — $O(n)$.

Idea 1: don't shift — move the front

Idea 1 — don't shift, move a head index: $O(1)$, but the queue crawls to the right



Keep a head index. Dequeue reads `block[head]` and adds 1 to head: $O(1)$.

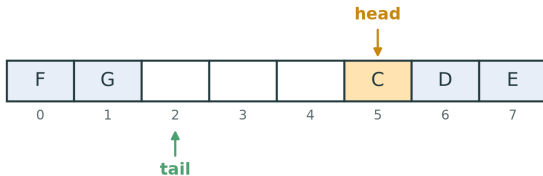
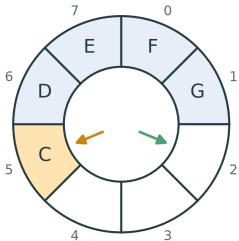
But the used part crawls right, and the slots it leaves behind are never used again.

The Ring Buffer

Idea 2: wrap around

Idea 2 — wrap around: the same fixed Array, read as a ring

head (amber) = 5 tail (green) = 2



after slot 7 comes slot 0: $i = (i + 1) \% \text{capacity}$

queue order, front to back: C D E F G
tail = (head + size) % capacity = (5 + 5) % 8 = 2

After the last slot comes slot 0 again:

- **front** at head; **back** at (head + size) % capacity
- every step forward is $(i + 1) \% \text{capacity}$

The two operations

enqueue(x)

1. full? — see the next slides
2. write x at $(\text{head} + \text{size}) \% \text{capacity}$
3. $\text{size} += 1$

dequeue()

1. empty? — `IndexError`
2. read the value at head; clear that slot
3. $\text{head} = (\text{head} + 1) \% \text{capacity}$; $\text{size} -= 1$

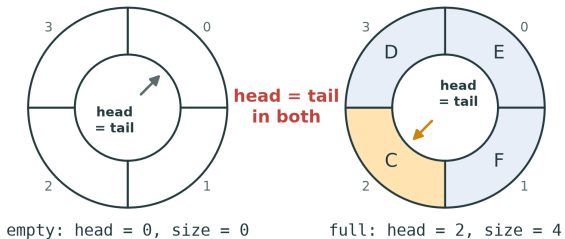
Nothing moves. Every step is a little arithmetic: $O(1)$.

Trace: capacity 4

Operation	Block	head	size	Returns
start	— — — —	0	0	
enqueue A, B, C	A B C —	0	3	
dequeue	— B C —	1	2	A
dequeue	— — C —	2	1	B
enqueue D	— — C D	2	2	
enqueue E	E — C D	2	3	← wrapped to slot 0
enqueue F	E F C D	2	4	← full
dequeue	E F — D	3	3	C

Full or empty?

Full or empty? The one real trap in a ring buffer



head and tail alone
cannot tell them apart.

Fix: keep size
(dsa/queue.py does),
or always leave
one slot empty.

When `head == tail`, the ring is **either** empty **or** full. Head and tail alone cannot tell which.

When the ring is full

Two honest choices:

- **Refuse:** raise an error. A *bounded* queue — fixed memory, like a real buffer.
- **Grow** (the challenge): allocate a bigger Array and copy — as `DynamicArray` does, doubling so that enqueue stays amortised $O(1)$.

Growing: the trap

Growing a ring: copy in QUEUE order, not slot order

full, capacity 4
head = 2



queue order: C D E F

naive copy
head still 2



reads C D _ _ x

unrolled copy
head = 0



reads C D E F ✓

```
new[i] = old[(head + i) % old_capacity] for i in range(size); then head = 0
```

Copying slot by slot keeps the wrapped order — so the queue reads **wrong**. Copy in **queue order**, and restart head at 0.

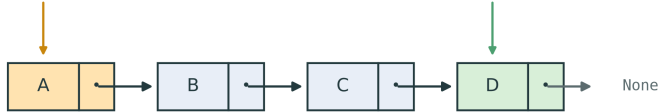
Two More Honest Queues

The linked queue

A linked queue: front at the head, back at the tail — never the other way round

head = front
dequeue: pop_front, $O(1)$

tail = back
enqueue: link after tail, $O(1)$



Front at the **head**, back at the **tail**: both operations $O(1)$ worst case.

A queue from two stacks

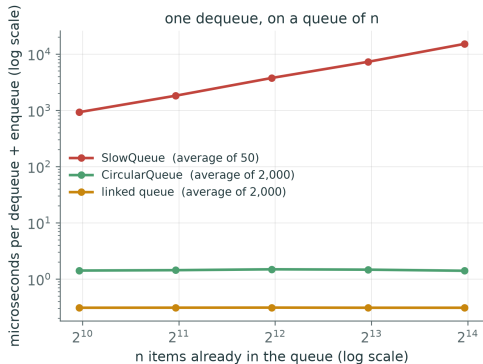
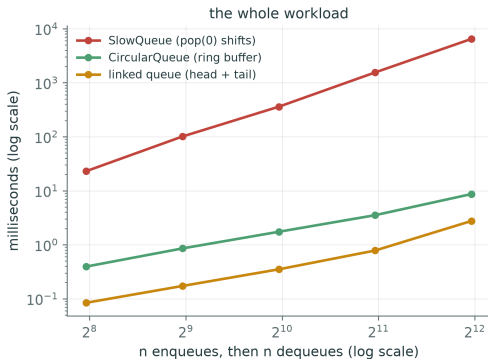
- inbox: enqueue pushes here.
- outbox: dequeue pops here. When outbox is **empty**, first pour **all** of inbox into it.

Pouring reverses the order — and LIFO reversed is FIFO.

Measured

The gap is the lesson

Measured: pop(0) makes each dequeue $O(n)$; the ring keeps it $O(1)$



Four queues, compared

	enqueue	dequeue	memory
SlowQueue — array, $\text{pop}(\emptyset)$	amortised $O(1)$	$O(n)$	compact
Array, head index, no wrap	amortised $O(1)$	$O(1)$	dead slots grow for ever
CircularQueue — ring buffer	$O(1)$ (amortised if it grows)	$O(1)$	compact, reuses slots
Linked, head + tail	$O(1)$	$O(1)$	a node per item
Two stacks	amortised $O(1)$	amortised $O(1)$	two arrays

Where Queues Are Used

FIFO is fairness — and breadth

- **Scheduling:** print jobs, requests to a web server, processes sharing a CPU (round-robin).
- **Buffers:** keyboard input, network packets, audio — ring buffers between a producer and a consumer; “the last k values” for a moving average.
- **Breadth-first search** (Week 14): visit everything 1 step away, then 2 steps, ... — the queue keeps the frontier in order.
- **Level-order traversal** of a tree (Week 11): the same idea on a tree.

The deque: both ends, both ways

A **deque** (double-ended queue, say “deck”): add and remove at **both** ends, all $O(1)$.

- only one end used → a **stack**
- add at one end, remove at the other → a **queue**

In Python: `collections.deque` — `append`, `appendleft`, `pop`, `popleft`.

This Week

Exercises: dsa/queue.py

Method	Target	The trap
SlowQueue.enqueue	amortised $O(1)$	append on self._items
SlowQueue.dequeue	$O(n)$ — on purpose	IndexError when empty; pop(0)
CircularQueue.enqueue	$O(1)$	tail = (head + size) % capacity; full?
CircularQueue.dequeue	$O(1)$	IndexError when empty; head wraps
challenge: grow when full	amortised $O(1)$	copy in queue order; head = 0

```
pytest tests/test_stack_queue.py -v -k queue
```

Homework 7 — before Lecture 08

1. **Implement** `dsa/queue.py` until the queue tests pass — then the challenge: grow instead of raising, and test it on a ring that has wrapped.
2. **Measure** `SlowQueue` against `CircularQueue` in `notebooks/07-queues.ipynb` and plot both. Estimate how long the `SlowQueue` would take for $n = 10^6$.
3. **Trace** a `CircularQueue(capacity=3)` through: enqueue 1, 2, 3; dequeue; enqueue 4; dequeue; dequeue; enqueue 5, 6. Show the block, head and size after each step.
4. **Design**. A queue from two stacks: write it, and explain why it is amortised $O(1)$ (W7-C1).

Summary

Seven things to keep

1. A queue is **FIFO**: enqueue at the **back**, dequeue at the **front**, both $O(1)$.
2. Empty dequeue raises **IndexError**.
3. On a plain array, one end is always the front: `pop(0)` or `insert(0)` — $O(n)$.
4. The **ring buffer**: head moves, nothing shifts; `tail = (head + size) % capacity`.
5. `head == tail` means **full or empty**: keep size (or waste one slot).
6. Growing a ring: copy in **queue order**, then `head = 0`.
7. Linked queue: front at the **head**, back at the **tail**. Two stacks: **amortised** $O(1)$.

Week 8 — Searching (*midterm window*). Seven ways to find a value, and what a **sorted** array buys you: from $O(n)$ to $O(\log n)$.